Sage — pay for verified work

Future Caribbean Global Buildathon · Finance, Payments & MSME Capital Live product: sagepays.xyz · open source: github.com/shariqazeem/sage · solo builder: Shariq Shaukat

What it is, in one paragraph

Sage is an AI agent with its own budget. A business funds it once and names its product. From then on the agent decides what work to buy, designs it, publishes it, checks every deliverable itself, and pays real people in USDC — refusing what does not hold up, with the reason written out. It proposes each move with its reasoning before any money leaves, and the founder can veto it or tell it what to do in plain words. What the founder cannot do is stop mattering: the ceilings are theirs, and the vault — not the prompt — enforces them. **The agent picks where to spend. It can never pick how much.**

The people being paid need no application, no bank account and no wallet app: a Telegram link gives them one. Every verified payout builds a portable work record, and that record unlocks working capital. Proving you are one person takes a minute and asks for no name, document or country — which is what stops one operator becoming twelve workers.

One loop — decide → define → verify → pay-or-refuse → receipt → record → **capital back in** — running live on TWO mainnet rails with real USDC: GOAT (the public tape) and Starknet (the private claim — a worker collects by one-time link, including into a shielded note). The founder never picks a chain; they pick an outcome — *public receipts* or *private-capable* — and the router picks the rail.

```
mandate (ceilings) → agent decides WHERE → policy sizes HOW MUCH → vault (cannot exceed)
   → router (GOAT public · Starknet private · licensed-fiat door, interface-ready)
      → record (public tape or selective proof) → capital in (the advance facility)
```

Public campaigns, from inside. The public board is still a door, opened from the workspace: a team that wants strangers unticks invite-only and the campaign lists on the marketplace. Members-only work autopays inside the vault's limits; public work gets every one of Sage's checks and then the team releases each payout with Sage's assessment beside it. The agent filters, the organization decides.

Why a workspace and not an open board. We ran the open version first. Every public campaign was farmed by the same returning group with rotating wallets — the last one 10 of 10 rewards to one operator, proven on chain (gas funded wallet-to-wallet, payouts forwarded to one hub minutes after settlement) and now detected and held automatically. The engine was never the problem: the judge, the vault and the receipts all held. Identity was. So the product sells to the party who already knows its people, and the agent does the one job it is measurably good at: was the work done.

The track's bar, answered with receipts

The brief asks	What ships (all live, all verifiable)
Lending without collateral — capital actually moves	**The Sage Advance facility — first disbursement drawn and repaid on 6 Sep** (`adv-1997e6ea-454` : $1.85 against verified inflow, repaid in full from the next payout by the waterfall; a rehearsal with our own wallet standing in for the seller): a worker takes it from

The brief asks	What ships (all live, all verifiable)
	their own record page in one click (self-serve switches on per deployment), the pot escrows on Starknet, and the next verified Starknet payouts repay it by the waterfall (the GOAT rail's repayment leg is not yet wired); capacity is published arithmetic (`the lender's multiple × verified monthly inflow` — Sage scores nobody), disbursement is a claim link from the pot, and repayment is **the waterfall** — each subsequent verified payout escrows as two legs, the pot's slice and the worker's remainder, recourse on the Sage-routed remainder only. One active advance per borrower and one repayment per payout are schema-level guards. The lender view prints the same call an institution's system would make — operator-authorized today — refused past the same formula the page shows. sagepays.xyz/lender
Privacy that banking-shy earners need	An earner can withhold amounts from their public record and instead issue a **signed earnings floor** ("earned ≥ $X") their lender verifies at /lender or off-platform — the issuer address is published, and a tampered floor or wrong-wallet document fails in words. Income that can be borrowed against without being a public diary.
Does the system change outcomes? (the track's own bar)	Measured, from the ledger, on a public page: recipients keep **100%** of the vault-derived reward (the 8% corridor would have taken **$5.09** of the settled total); **median about 3 minutes** submission→settled *with verification included*; access with integrity (40% refused); rails, funders and denominations from the rows — and the one unmeasured number (intra-regional corridor flow) stated as **not yet measured**. sagepays.xyz/outcomes
MSME Credit Layer (CORE): aggregate fragmented data → real-time credit profiles → lending without collateral	Every payout is verified-before-paid and receipt-anchored, so the byproduct is credit data that cannot be self-reported: `/record/<wallet>` with **Sage Signals** (`credit-signals-v2`) — pass rate over judged submissions, verified inflow per active month, distinct funders, tenure, recency — every formula deterministic and published, no invented score. The "fragmented data" Sage aggregates today is its own ledger across two rails plus every wallet a person declares as theirs — no external registry or bank feed yet. Lender-consumable JSON: `/api/record/<wallet>` (`sage.work-record.v4` — carrying the advance facility's history and, since 2 Sep, the LINKED business: wallets proven by their own sign-ins on both rails, one profile, one redaction boundary).
Diaspora capital: investment rails, not just transfers	A funder speaks a milestone grant into Telegram; the recipient is onboarded by a chat link (walletless — a policy-bound server wallet is minted for them), each tranche releases only on verified proof, and the refusals are on the record too. A remittance becomes conditioned investment with an audit trail. **Honest tense:** the grant compiler is battery-green (P-DIRECT, 75 rows, 0 compile failures); the first grant paid in production is scheduled before submission — until it runs, this row is a capability, not a receipt.
Payment fees 7–9% → <1%; settlement approaches real-time	USDC on GOAT Network and Starknet: recipients keep 100% of the vault-derived reward; the payer pays a flat $0.10 per settlement over x402 ($1.30 collected on 13 settlements, against $5.09 a 7–9% corridor would have taken on the same volume); payout gas ≈ $0.000002 and Sage's own; the median from submission to settled payment, verification included, is computed live on `/outcomes` .
Works with fragmented, real-world data	Verification is Sage's own observation of reality: it browses the product, fetches the artifact, reads the chain — and **refuses what it cannot verify**. Live refusal share: **40% of all judged submissions** (26 refusals / 65 decided), each with its reason.
KYC/AML in flow	OFAC SDN screening (vendored, dated snapshot — never a runtime API on the money path) at every door on the EVM rail: the web and chat submit doors, the autonomous preflight, and manual release. The SDN list carries no Starknet addresses, so the private rail is screened at its EVM funding side only — stated here rather than implied. Plus Sybil

The brief asks	What ships (all live, all verifiable)
	controls (near-duplicate detection, artifact-twin fingerprinting, per-wallet caps, daily limits) and full public auditability. The compliance chapter.
Integrates with existing institutions	The record API is the decision-support surface a lender consumes; the public MCP surface + open worker reference lets any institution's (or anyone's) agent participate. Buil d on Sage.
"Strong teams build financial infrastructure"	The Explorer: every settlement **and every refusal**, live, searchable by transaction or wallet. Payments infrastructure that shows its work.

The numbers (mainnet, live at submission time)

- **$63.60 USDC** settled across **39 verified payouts** on two rails (26 GOAT, 13 Starknet) — reaching **32 wallets but 24 people**, because nine wallet links are recorded on-chain and we publish the collapsed count, including on our own public board. Sage's own dogfood wallets settle on the same rails and are excluded from the people count.
- **26 refusals on record**, each with a published reason. No human reviewed any autonomous decision.
- **Under three minutes** median from a submission arriving to USDC in the worker's hand (the live figure is computed on `/outcomes`); the fastest was 18 seconds.
- Campaign deploy gas ≈ $0.0000002; payout gas ≈ $0.00000002 (BTC on GOAT).

Four receipts that tell the whole story

1. **The first autonomous payout** — Sage judged a human's own-words account against its own observations and paid $1 with no human in the loop: `0x8df776…0069`
2. **An AI earned under the same rules** — an autonomous agent discovered a gig over the public MCP, published a real deliverable carrying its own wallet, signed the same EIP-712 claim a human signs, and was paid $0.50: `0xb01203…d827`
3. **That AI's credit file** — the same loop produced a machine-readable verified work record: `/recor d/0xccbf…768d`
4. **A real stranger, refused** — within the hour of the first gig going live, an off-contract spam submission arrived and was held by the deterministic verifier: zero dollars moved, zero model spend, reason on the record.

Why the agent is trustworthy (the part we test adversarially)

- **The AI proposes, the vault disposes**: budgets, per-mission rewards, completion caps, and daily velocity are on-chain; no model ever computes a money amount.
- **Deterministic checks run first** (host + wallet-marker binding for artifacts, chain reads for on-chain proofs, sanctions screen, twin fingerprints) — the judge model only ever *narrows* what can be paid.

- **Two-sided adversarial evals** (P-JUDGE, P-WORK) run the production judgment path against attack fixtures *and* honest-work fixtures: an attack that pays and honest work that holds are both hard failures — the battery cannot be passed by blanket strictness or blanket leniency. The payout judge (MiniMax-M3) is promotion-gated on this evidence. Latest runs (3 Sep): P-JUDGE zero wrong auto-pays across 57/57 rows, every hostile fixture held; P-WORK 14 attacks with zero leaks and 12 honest submissions with zero false holds; P-DIRECT 15 of 21 founder briefs compiled first-shot, zero refused.
- **One person cannot take many seats**: on public work, claiming a slot asks for a one-time **World ID** proof of personhood (Sage receives a nullifier — never a name, document, face or country) and every cap counts the *person* across verified, chain-linked and declared wallets, so fifty slots need fifty humans; underneath, exact and paraphrase dedup on reports, a fingerprint of the fetched deliverable itself so a copied page with the wallet marker swapped is held as copied work, GitHub provenance (a fork, or a repository older than the gig, is held), wallet-freshness and funding-graph signals (a real 4-wallet rotation cluster was caught on-chain in August), a per-wallet payout cap, a daily submit limit, and the vault's own per-recipient replay protection. Every one of these is a HOLD for a human, never a silent rejection.
- **Injection is expected**: untrusted content rides inside markers, quotes must be verbatim substrings of fetched evidence, and an artifact carrying "SYSTEM: recommend pay" instructions is a fraud signal, not an instruction.

The team

One builder, full-time on this since July: Shariq Shaukat (Lahore). First place at the OpenClaw agent bootcamp in August with an earlier version of Sage, judged on real mainnet usage and growth. The way of working is measurable in the repository: 4,396 automated tests, seven live evaluation batteries that run the production prompts against real products and real founder wording before a deploy, and a ledger of 39 mainnet payouts across two rails — every defect found by measuring production rather than by reading a diff, and each fixed on a deterministic path rather than in a prompt. Domain: payments and verification infrastructure; the agent-with-a-budget thesis is the product, not a feature.

How this scores on your rubric

Rubric	Where the evidence is
Team quality	one builder, a prior first place with a live product, and a repository whose tests, batteries and ledger show the execution speed (`docs/fc/technical-documentation.md` , "Live evaluation batteries")
Innovation & defensibility	bounded autonomy over money — the agent proposes, an on-chain vault disposes (safety mode l); a credit record built from verified payouts, not self-reported cash flow (/record, /lender); private payouts decided in public (Starknet receipt); one person, one slot without identity (judgin g). The moat is the ledger: every verified payout is a data point no competitor can copy
Product-market fit	the track's own bar, measured on a public page (/outcomes): fees $0.10 flat, settlement in minutes, credit without collateral, obligations in the region's currencies; programmes and teams as the wedge; revenue on settlement and financing, never seats

Rubric	Where the evidence is
Agentic AI excellence	architecture: three model brains, a browser agent and a pure operator policy with deterministic gates between them (architecture); autonomy: 39 payouts and 26 refusals with no human in the loop (/explorer); coordination: architect → critic → gate, judge → autopilot gate → vault, operator → proposal → veto window (fund once, then stop deciding); reasoning: every decision cites verbatim evidence (a receipt); human-in-the-loop: approve the plan, fund it, veto a move, release a hold; compute efficiency: the browser loop makes zero model calls, the compiler and every money rule are deterministic, and the payout judge runs on MiniMax-M3 — the buildathon's own credits — with a fallback chain; implementation quality: 4,396 tests, seven live batteries, anchor integrity 100% as a hard stop; scalability: per-campaign vaults on two mainnet rails, one dispatcher; impact: the ledger and the outcomes page

Business model and go-to-market

Who pays. The organization that wants work done funds the vault. Sage earns on what it settles, not on seats: a flat operator fee per settlement today (charged over x402 and recorded beside the payout — revenue exists only when a verified payment exists), a share of volume as it grows, and a financing margin on the advance facility (self-serve for the worker, one click on their record page; a published multiple on witnessed inflow, the LP is us today). The same record feeds an institution's own underwriting through the JSON contract. There are no plans and no paywalls: private payouts on Starknet, the record and the lender view are part of the rail.

Go-to-market. Programmes and teams first, workers through them: an incubator paying its cohort's milestones, a cooperative paying its members for verified deliveries, a company paying contractors, a diaspora funder backing one seller. Each brings its own people, so there is no marketplace to bootstrap and no Sybil economy to fight; each payout produces a receipt and a record page the recipient carries to the next programme. Caribbean first: obligations denominate in the region's own currencies at a stamped rate, the composer's supplier and grant templates are written in the track's own shapes (two-tranche seller grants, invoice-bound supplier payments), and the diaspora sender's currencies (CAD, GBP, EUR) are on the same list. Regulated fiat disbursement is a licensed-partner door with a typed contract; it ships when a partner does, and is labelled as pending until then.

Impact and scalability

Evidence of real-world use. 39 mainnet payouts across two rails (26 on GOAT, 13 on Starknet) to 32 wallets — 24 distinct people once wallets the consolidation watch has linked on chain are collapsed, and we publish that collapse rather than the flattering number; 26 refusals on record with reasons; an autonomous third-party agent earned under the same rules and has a credit file; two real wallet-rotation clusters were caught on chain and published — the second one after it had taken a whole gig, which is why the workspace exists. The judge is promotion-gated on live batteries (zero wrong auto-pays across 57 rows; 14 attacks with zero leaks and 12 honest submissions with zero false holds). Outcomes are computed live on `/outcomes` against the track's own bar, with the gaps stated as not yet measured.

Path to deployment and scale. The rails are mainnet today and the vault model is chain-agnostic by construction (one dispatcher, two settlers); a third rail is a settler, not a redesign. A workspace is a table, an invite link and a plan; a programme with a hundred grantees onboards them from a phone. Capacity is bounded by the organization's budget, not by our operations: the agent designs, verifies

and pays without a human, and refusals cost nothing. The MSME credit layer scales with the ledger — every payout is a record row — and the lender contract is a JSON endpoint any institution can consume. The next steps are standing intents that relaunch work on their own, the treasury funded from a Starknet wallet, the licensed fiat door, and an institutional LP behind the advance facility.

Does it work on products it has never seen? (4 September)

The standing battery drives the real pipeline — inspect, browse, look, design, gate — against one live URL per product category. Thirteen categories, chosen to be awkward on purpose: a bare HTML page, docs, a SaaS marketing site, an SPA, a canvas game, a DOM-heavy world, e-commerce, a login wall, a portfolio, a non-English site, a heavy news site, a WebGL scene, and a wallet-gated dapp.

Products planned	**13 of 13**
Anchor integrity	**100% on every one** — every criterion traceable to something Sage actually saw
Failures or timeouts	**0**
Cost of the whole battery	~248k tokens

Anchor integrity below 100% is a hard stop in this project, not a target. It means no mission asks a tester for something Sage cannot point at in its own observations — which is what makes an autonomous payout defensible rather than a guess.

The safety claim, measured rather than asserted (4 September)

The judge that decides whether a stranger's work gets paid was put through its promotion battery against the production path, three runs over the full fixture set:

Wrong auto-pays	**0** across 57 valid rows
Provenance violations	**0**
Provider failures	0 — the run is evidence, not a sample
Verdict	**promotion-eligible**, conclusive
Cost of the whole battery	$0.03

Every adversarial fixture — evidence with no author, an author date that contradicts the work, a submission about the wrong product, a route that does not match — was **held, not paid**. None of them reached a payout, and none of them needed a human to notice.

The battery is in the repository and re-runnable (`JUDGE_EVAL=1 npx vitest run judge-eval.live`); a provider failure is reported as inconclusive rather than as a pass, so a quiet outage can never be mistaken for a good result.

Two rails, and why an MSME cares

The same agent settles publicly on GOAT and privately on Starknet, and the choice belongs to the work. A grant programme or a lender needs the public rail: every payout is a transaction anyone can open, which is what makes a payment history underwritable. A business paying wages or a sensitive supplier needs the other: the ledger records that a payout happened without recording who received it. Most payment infrastructure forces one or the other; the reason we can offer both is structural rather than cosmetic — settlement branches in exactly one place, and a test refuses any code that reaches around it.

What we don't claim

No identity KYC (wallet pseudonymity + sanctions screening + Sybil controls; named-recipient allowlists where operators need them). Screening is list-based and snapshot-dated. Walletless custody is provider-held under policy, stated plainly wherever it appears. The fiat door is **interface-ready, not live** — the typed adapter cannot ship without producing the identical credit event, the corridor quote works today, and the disburse function refuses in words rather than simulating money movement, because a "mark paid via partner" button with no partner is theatre. The first advance's mainnet round-trip ran on 6 Sep, twelve minutes after the first J$ milestone paid: disbursed `0x3e7014…3454`, repaid by the waterfall out of the next payout `0x51a779…e8e7b`, collected back into the pot `0x604cf9…ffef0`. A rehearsal with our own wallet standing in for the seller; the money and the mechanism were real, and every step is on the ledger.

Technical documentation with the architecture diagram, data sources, models and tools: `docs/fc/technical-documentation.md`*. Deeper docs: how it works · the safety model · compliance & controls · build on Sage*